VINUNIVERSITY

Fine-tuning LLMs — Từ Full Fine-tune đến LoRA/QLoRA

AICB-P2T3 · Ngày 21 · Chương 5 — Fine-tuning & An Toàn

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 5

HÃY SUY NGHĨ...

“Bạn có 5.000 hội thoại chăm sóc khách hàng bằng tiếng Việt. Prompt engineering đã hết đường — bây giờ làm gì?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội dung bài học — 5 phần, 25 module

I Quyết định

1. Có nên fine-tune không?

II Model đến từ đâu

2. Vòng đời model & các lab frontier
3. Pre-training
4. Mid-training
5. Post-training
6. CPT vs SFT vs Distillation
7. Optimizer & kiến trúc

III Lõi fine-tuning

8. Full fine-tune & bộ nhớ
9. Bản đồ PEFT
10. LoRA — cơ chế
11. LoRA Without Regret
12. Biến thể LoRA
13. QLoRA & lượng tử hoá
14. Siêu tham số & vòng lặp
15. Tăng tốc: kernel, loss
16. Công cụ & dịch vụ

IV Dữ liệu

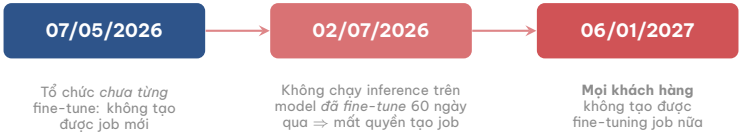
17. Dataset, template & masking
18. Dữ liệu tổng hợp
19. Dữ liệu trajectory (agent)
20. Data engine & dữ liệu chuyên gia

V Ra sản phẩm

21. Đánh giá
22. Thất bại & debug
23. Merge, deploy & serving
24. License & pháp lý
25. Demo & Lab 21

Bối cảnh 2026 — OpenAI thu hẹp fine-tuning self-serve

OpenAI thông báo ngừng nền tảng self-serve fine-tuning



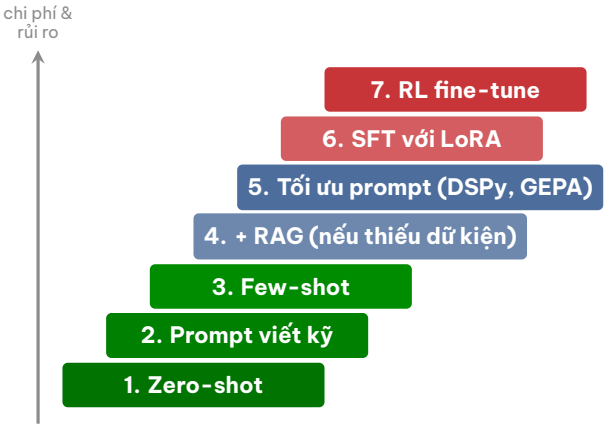
Điều gì vẫn còn

Inference trên model đã fine-tune chạy **đến khi base model bị khai tử** — và `o4-mini` (đường RFT) tắt ngày **23/10/2026**. Managed FT *chưa* chết: Vertex (Gemini SFT; RFT preview *chỉ để thử nghiệm*), Bedrock (RFT cho gpt-oss, Qwen3), Together, Fireworks (§16).

Lưu ý: Không flagship nào ra mắt 09/2026 (Claude 5.5, GPT-6.1 Sol, Gemini 4 Argon, Grok 4.7) cho khách fine-tune. **Tự fine-tune model mở** mới là con đường bền vững — nội dung hôm nay (§2).

Nguồn: developers.openai.com/api/docs/deprecations (kiểm 06/10/2026); trang model gpt-6.1-sol: Fine-tuning "Not supported".

Bậc thang baseline — leo từng bậc, đo ở mỗi bậc



Mỗi bậc *phù hợp với bài toán* là một **baseline cần đo** trước khi bước lên bậc trên. Chỉ bậc 6–7 (**đỏ**) mới đổi trọng số.

Bậc 5 mạnh hơn bạn nghĩ: GEPA — tối ưu prompt bằng cách đọc lại trajectory lỗi — thắng RL (GRPO) **6% trung bình**, tới 20%, và dùng rollout ít hơn **tới 35 lần** — trên 6 tác vụ được nghiên cứu.

Không phải lối đi bắt buộc

Dữ kiện hay đổi thì RAG lên sớm. Thiếu năng lực chung thì thử model *lớn hơn* trước khi fine-tune model nhỏ.

Cây quyết định 2026 — sáu câu hỏi và một cổng



Từ khoá: **SFT** học từ câu trả lời mẫu · **LoRA** chỉ cập nhật adapter nhỏ · **RL** học từ tín hiệu thưởng · **verifier** bộ kiểm kết quả tự động · **bộ eval** = metric, holdout, ngưỡng hồi quy, chi phí/độ trễ — chốt *trước* (§21).

Khi nào fine-tuning có lãi? — tính thử một lần

Bài toán: 1 triệu request. **A:** Gemini 3.8 Flash + prompt dài. **B:** Gemini 3.1 Flash-Lite đã tune, prompt ngắn. *Giả định* cùng chất lượng.

	A: prompt	B: tuned
Token vào/ra mỗi request	1.600/200	300/200
Giá vào/ra ($/1M token)	0,75/3,75	0,375/2,25
Chi phí / 1M request	**$1.950**	**$562,50**

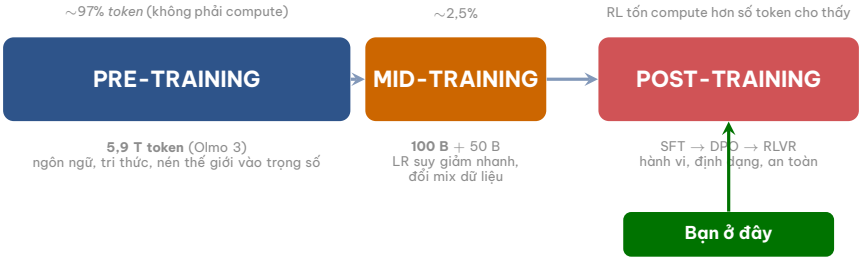
Tiết kiệm **$1.387,50** mỗi triệu request. Train 500 mẫu × 500 token × 3 epoch = 750K token = **$2,25**. Tiền train gần như bằng 0 — *tiền thật nằm ở dữ liệu và đánh giá.*

```
hoa_von = F / (cost_A − cost_B)
```

F: dữ liệu, eval, người	Hoà vốn
$500	360K request
$5.000	3,6M request
$5.000, giá 2027	1,5M request

Lưu ý: Đáp án đổi 10 lần theo **F** — con số bạn hay đoán sai nhất. Và endpoint tuned của Gemini tính **gấp 1,5** giá base.

Bản đồ ba giai đoạn — và vị trí của bạn



Vì sao bản đồ này quan trọng với người fine-tune

Fine-tuning **không phải một kỹ thuật riêng biệt** — nó là *giai đoạn cuối cùng của cùng một đường ống*, chạy trên ngân sách nhỏ hơn hàng nghìn lần. Mọi kỹ thuật ở §5–§6 (SFT, distillation, RL) đều là thứ các lab đã chạy trước bạn, chỉ khác quy mô. Hiểu hai giai đoạn đầu để biết mình *thừa hưởng* gì — và cái gì đã *không còn sửa được* bằng fine-tune.

Model mở 10/2026 — cỡ bạn thật sự fine-tune được

Model (ngày)	Tổng / kích hoạt	License	Ghi chú cho người fine-tune
Qwen3.5-4B (02/2026)	4B dense	Apache 2.0	**model của Lab 21**; đa phương thức
Qwen3.8-27B (08/2026)	27B dense	Apache 2.0	dense, dễ tune nhất ở cỡ này
Gemma 4 (04/2026)	dense $\leq$31B; 26B-A4B	Apache 2.0	Google: "tune and deploy" thương mại
Muse Glimmer (08/2026)	30B dense	Apache 2.0	Meta; agentic; SFT + OPD + RL
Granite 4.2 (08/2026)	3B, 8B, 30B (dense)	Apache 2.0	gọi hàm kiểu OpenAI; RL sandbox
Nemotron 3.5 Lightning (08/2026)	30B / 3B	OpenMDW 1.1	hybrid Mamba-2; recipe LoRA & RL
SEA-LION v4.8 (09/2026)	30B / 3B	MIT	**có tiếng Việt**; *chưa căn chỉnh an toàn*
gpt-oss-20b (08/2025)	21B / 3,6B	Apache 2.0	OpenAI; Bedrock RFT hỗ trợ
Olmo 3 (11/2025)	7B, 32B (dense)	Apache 2.0	mở cả dữ liệu & checkpoint

Lưu ý: "Open weights" $\neq$ Apache: Qwen3.8-Flash-Next, Kimi K3, MiniMax M3 có license *riêng*; Mistral Medium 3.5 trên $20 triệu/tháng cần giấy phép thương mại (§24).

Chọn base theo thứ tự này

License → ngôn ngữ (tiếng Việt?) → cỡ vừa GPU (§13) → dense trước MoE → chat template & tool format.

Lab đóng — họ kể gì về cách train, và cho bạn tune gì

Lab	Công bố về training	Chữ ký post-training	Khách fine-tune được (10/2026)	Trọng số mở
Anthropic	pretrain rộng; "substantial post-training" theo constitution	căn chỉnh theo constitution	không tìm thấy đường public cho Claude 5.5	—
OpenAI	thành phần dữ liệu pretrain + chấm điểm trong RL	RL grading, deliberative alignment	self-serve thu hẹp (§1); Azure: GPT-4.1, o4-mini RFT	gpt-oss
Google	model card thưa; tài liệu tuning chi tiết	—	Gemini 3.5 Flash / 3.1 Flash-Lite: SFT; RFT preview *không cho production*	Gemma 4
Meta	Muse Spark: scaling + RL; Glimmer: 3 giai đoạn rõ	SFT + OPD + RL	chưa có endpoint tune	Muse Glimmer
xAI	Grok 4.7: RL dài hơn, task khó hơn	RL trên task nhiều giờ	không có dịch vụ public	Grok 2, Grok 1
Mistral	sản phẩm; Forge cho doanh nghiệp	Forge: pretrain → RL	FT host cũ **đã deprecated**; Forge	Small 4
Microsoft	Phi: lọc, sửa lỗi, dữ liệu tổng hợp	mode token suy luận/không	Foundry: Phi-4, Phi-4-mini	Phi-4 (MIT)
Amazon	Nova Forge mở checkpoint từng giai đoạn	SFT, RLHF (Nova 1)	Bedrock: Nova 2 Lite SFT + RFT	—

Nhiều API flagship chưa cho fine-tune trực tiếp

Kiểm catalog từng dịch vụ tại ngày dùng. Mang về từ lab đóng: **phương pháp** (constitution, RL task dài, mode token).

Sáu thứ bạn thừa hưởng — và fine-tune không sửa được

Thừa hưởng	Fine-tune sửa được?
Tokenizer & vocab	× Gần như không
Kiến trúc, số lớp	× Không
Độ dài ngữ cảnh gốc	~ Chỉ bằng CPT tốn kém
Ngôn ngữ chưa được học	~ Rất tốn (§6)
Bias ẩn của optimizer	~ Xem §7
Hành vi, định dạng	✓ **Việc của bạn**

Quy tắc chọn base model

Chọn base theo thứ tự:

1. Đã thấy đủ *tiếng Việt* chưa? — đo perplexity trên văn bản miền của bạn *trước* khi huấn luyện.

2. License cho phép mục đích của bạn không (§24)?

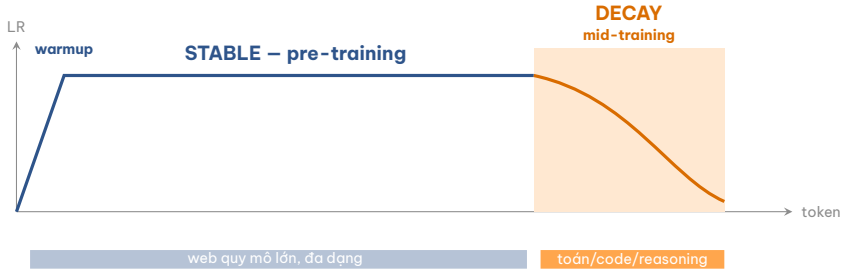
3. Kích thước có phục vụ nổi không (Ngày 20)?

Chỉ sau đó mới xét benchmark.

Một câu để nhớ

Fine-tuning **định hình lại** thứ mô hình đã biết, không **tạo ra** năng lực chưa có. SFT dạy được tri thức *hẹp*; 5.000 mẫu không biến base chưa thấy tiếng Việt y khoa thành giỏi *y học*.

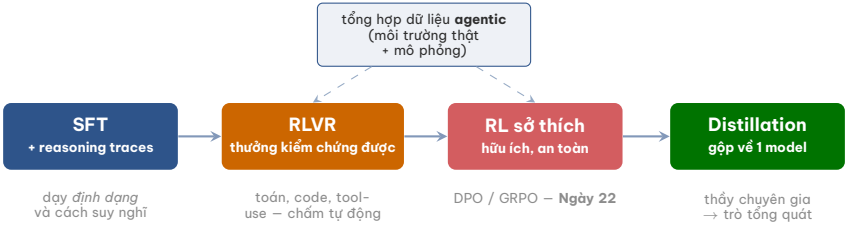
Định nghĩa & vị trí — đọc được đường learning rate



Hai thứ đổi *cùng lúc*: learning rate suy giảm nhanh **và** hỗn hợp dữ liệu chuyển sang chất lượng cao.

Mid-training — Giai đoạn *bắc cầu* giữa pre-training và post-training: dùng dữ liệu và compute trung gian để nâng năng lực cụ thể (toán, code, suy luận, ngữ cảnh dài) **trong khi vẫn giữ** năng lực nền. Thuật ngữ chưa thống nhất trong tài liệu — hãy định nghĩa trước khi dùng.

Bộ xương chung — và bạn đang làm bước nào



Bạn ở bước 1 — và đó là bước đúng

Với gần như mọi dự án doanh nghiệp, **SFT là toàn bộ công việc**. Ba bước sau cần hạ tầng rollout, hàm thưởng kiểm chứng được và ngân sách GPU của một lab. Biết chúng tồn tại để *đọc được tech report* và để biết khi nào mình đã chạm trần của SFT.

Ba công cụ, ba loại ngân sách

	Continued Pretraining	SFT	Distillation
Dạy điều gì	Tri thức, phân phối miền	Hành vi, định dạng	Bắt chước một teacher
Dữ liệu	Văn bản thô, không nhãn	Cặp có nhãn	Đầu ra của teacher
Đơn vị đo	**Token** (tỷ)	**Mẫu** (nghìn)	Lượt rollout
Mục tiêu	Next-token	Next-token có mask	Khớp phân phối teacher
Rủi ro chính	Quên thảm hoạ	Quá khớp	Lệch phơi nhiễm

Chúng nối tiếp nhau

Một checkpoint đã qua CPT là **điểm xuất phát tốt hơn** cho SFT: SFT theo miền hội tụ nhanh hơn và đạt chất lượng cao hơn khi khởi đầu từ CPT.

Lưu ý: Nếu vấn đề của bạn là "model không *biết*" thì vài nghìn mẫu SFT hiếm khi đủ cho một lỗ hổng *rộng* — so CPT và RAG bằng eval trước khi đổ thêm mẫu.

Quên thảm hoạ — và liều thuốc rẻ tiền

Quên thảm hoạ — Model học tốt miền mới nhưng **đánh mất năng lực tổng quát** đã có — toán, đa ngôn ngữ, tuân thủ chỉ dẫn.

Năm họ giải pháp:

- Chính quy hoá — phạt việc trôi khỏi trọng số quan trọng
- **Replay** — trộn lại dữ liệu phân phối gốc
- Model merging — nội suy giữa base và bản fine-tune
- Kiến trúc — **LoRA chính là họ này**: base đóng băng *là* bản sao được bảo toàn
- Chiếu trực giao — OPLoRA và tương tự

Tỷ lệ replay

1–5% dữ liệu gốc đã giảm quên rõ rệt với chi phí downstream không đáng kể.

Với continued pretraining, khoảng **5–20%** thường được dùng.

Ngay cả **1%** cũng đo được tác dụng.

Luật vận hành

Đo benchmark tổng quát *trước và sau*. Tụt **3–5 điểm** ⇒ tăng replay, hoặc chuyển sang LoRA.

Đổi optimizer lúc fine-tune — nay đã có câu trả lời

VINUNIVERSITY

Optimizer frontier có chạm tới người fine-tune không? Năm 2026 có hai bài báo trả lời thẳng — và chúng nói ngược nhau, vì **trả lời hai câu hỏi khác nhau**.

Pre-training (NVIDIA, 07/2026)

"SOAP và Muon liên tục vượt AdamW ở các quy mô chúng tôi thử" — model nhiều tỷ tham số, hàng nghìn tỷ token, batch tới **100M token**, nơi **AdamW suy giảm**.

arXiv 2607.20548. Phần tóm tắt **không** tuyên bố gì về fine-tuning.

Fine-tuning (05/2026)

*"Chuyển sang Muon để fine-tune một cách ngây thơ dẫn tới hiệu năng suy giảm do **lệch optimizer**"* — lệch này phá kiến thức đã pre-train và **tỷ lệ với độ mạnh của bước cập nhật**.

arXiv 2605.10468, "Can Muon Fine-tune Adam-Pretrained Models?"

Và đây là chỗ nó nối vào phần còn lại của buổi học

Khuyến nghị của chính bài báo đó: **dùng LoRA** khi buộc phải fine-tune bằng optimizer khác — *"LoRA thu hẹp khoảng cách giữa Adam và Muon quan sát được ở full fine-tune"*. Ràng buộc độ lớn cập nhật **chính là thứ hấp thụ độ lệch**. Một lý do nữa để LoRA là mặc định nó thay thế cho những thứ full FT không thay

Kiến trúc 2026 — thưa hơn, và attention không còn là một thứ

VINUNIVERSITY

Độ thưa đã thành cực đoan

Model	% tham số kích hoạt
DeepSeek-V4-Pro	≈3,1%
Kimi K3	≈3,7%
Qwen3.8-2.4T	≈4,0%
GLM-5	≈5,4%

Kimi K3: 896 expert, chọn 16 + 2 dùng chung, 93 lớp. DeepSeek-V4 thêm **mHC** — ràng buộc residual lên một đa tạp để khôi phục ánh xạ đồng nhất.

Bốn họ attention cùng tồn tại

Họ	Ví dụ 2026
Softmax thưa/nén	DeepSeek **CSA** + **HCA**, **DSA**
Linear lai	**Gated DeltaNet** (Qwen3-Next, 3:1); **KDA** (Kimi K3)
Local/global xen kẽ	Gemma 4 (4:1 và 5:1)
GQA + RoPE thuần	model dense nhỏ — *thứ bạn thật sự fine-tune*

Mục tiêu chung: giữ trạng thái **kích thước cố định** để chi phí giải mã mỗi token không tăng theo độ dài ngữ cảnh.

Fine-tune model MoE — "ở đâu" giờ gồm cả "expert nào"

Định tuyến expert **lệch nặng**: một nhóm nhỏ expert nhận phần lớn token mỗi lớp. LoRA mặc định lại gắn adapter vào *mọi* expert — phần lớn adapter nằm chờ ở expert gần như không được gọi.

Định tuyến trước, rồi mới gắn adapter

Đếm số lần định tuyến trên một tập hiệu chuẩn nhỏ, chỉ gắn LoRA vào **top 25% expert được gọi nhiều nhất mỗi lớp**:

- chênh lệch chất lượng **trong ±1 điểm %**
- tham số huấn luyện **−70…73%**
- checkpoint adapter **−71…73%**
- thời gian huấn luyện **tới −50%**

Lưu ý: **Tín hiệu định tuyến mới là thứ tạo ra kết quả**, không phải ngân sách tham số: chọn expert **ngẫu nhiên** với cùng số tham số cho kết quả **kém hơn ≈2,5 điểm %**.

Hai lưu ý vận hành:

- **Đừng huấn luyện lớp router** — nhà cung cấp tắt mặc định vì lý do ổn định.
- Cùng một phép toán, khác bộ nhớ: PEFT *hiện thực hoá* delta LoRA cho mọi expert trước phép nhân MoE; Unsloth đổi thứ tự phép toán để tránh việc đó.

Đây là mở rộng tự nhiên của kết quả ở §11.2 — *vị trí* gắn adapter quan trọng hơn *rank*. Trên MoE, "vị trí" nay bao gồm cả "expert nào".

Khối khuếch tán — model không còn sinh từ trái sang phải

DiffusionGemma (Google DeepMind, 10/06/2026, Apache 2.0, **26B MoE / 3,8B kích hoạt**) là model ngôn ngữ **block-diffusion** có trọng số mở đầu tiên đủ lớn để fine-tune thật.

Cách nó sinh chữ • prefill: attention **nhân quả** đọc prompt
• sinh: attention **hai chiều** khử nhiễu lặp một **"canvas" 256 token** — *song song*
• canvas sạch → commit KV cache → block kế Tới **4**×

nhanh hơn khi sinh — 700+ tok/s trên RTX 5090, 1000+ trên một H100.

Ngày đầu: vLLM, Transformers, SGLang, MLX.

Công thức SFT — ba bất biến của deck *còn*, một *đổi*

Bất biến	Số phận
Vị trí > rank (§11.2)	**còn** — LoRA $r = 16$ trên attn + dense-MLP
Đừng train router (§7.5)	**còn** — "router MoE giữ đóng băng"
Merge rồi kiểm (§23)	**còn** — vẫn là PEFT
Che loss ở câu trả lời (§17)	**ĐỔI**

Vì sao "che loss" không còn nghĩa như cũ

Loss là **CE trung bình trên token được giám sát trong khối được chọn**, cộng một **encoder AR loss** trên chuỗi sạch ($t \sim U(0{,}001;\ 1)$). Không có "đoạn trả lời" trái-sang-phải để che — giám sát do **khối** và **lịch nhiễu** định nghĩa. Kỹ năng NB1 vẫn đúng; *cách* kiểm thì khác.

Kế toán bộ nhớ — 16 byte cho mỗi tham số

Thành phần	Byte/param	Ghi chú
Trọng số (bf16)	2	bản đang tính
Gradient (bf16)	2	mỗi tham số một gradient
Adam *m* (fp32)	4	momentum bậc 1
Adam *v* (fp32)	4	momentum bậc 2
Master weights (fp32)	4	bản chính xác cao
Tổng	**≈16**	**chưa tính activations**



Quy tắc nhẩm — **≈16 GB trạng thái cho mỗi tỷ tham số**, cộng thêm activations.

Đây là lý do LoRA hiệu quả đến thế: adapter chỉ chiếm ∼1% tham số, nên **ba hàng fp32 ở giữa co lại theo tỷ lệ đó** — phần đắt nhất biến mất.

Khi nào Full FT vẫn là lựa chọn đúng

VINUNIVERSITY

Chọn Full FT / CPT khi

- Dữ liệu quy mô **tỷ token** (CPT — §6)
- Dịch chuyển miền rất lớn: ngôn ngữ mới
- **Đổi tokenizer / mở rộng từ vựng**
- Bạn đang *tạo ra* một base model

Chọn PEFT khi

- Quy mô dữ liệu **post-training** thông thường
- Cần nhiều biến thể cho nhiều khách hàng
- Muốn giữ năng lực gốc (§6 — quên thảm hoạ)
- Ngân sách GPU thực tế

Điều đã thay đổi trong 2025–2026

Trước đây mặc định là "full FT tốt hơn, LoRA là bản rẻ tiền". Nghiên cứu 2025 cho thấy **trong vùng dữ liệu post-training, LoRA cấu hình đúng đạt *cùng* chất lượng** — phần lớn bằng chứng cũ đến từ *cấu hình sai* (§11).

“Thuật toán tối ưu” — một cái nhãn, bốn loại việc khác nhau

Danh sách nổi tiếng nhất trong hệ sinh thái (LLaMA-Factory) gộp 12 cái tên vào một dòng. Chúng **không cùng loại** — và phân loại lại chính là cách nhớ được chúng:

Loại	Đổi cái gì	Thành viên
A. Full FT tiết kiệm bộ nhớ	**optimizer**	GaLore · BAdam · APOLLO · Adam-mini · Muon
B. Hình dạng cập nhật	**tham số hoá** LoRA	DoRA · LoRA+ · OFT/OFTv2 · rsLoRA · GraLoRA
C. Khởi tạo	adapter **bắt đầu ở đâu**	PiSSA · LoftQ · OLoRA · EVA · CorDA
D. Phẫu thuật kiến trúc	**chính model**	LongLoRA · LLaMA Pro · Mixture-of-Depths

Lưu ý: **Loại D là cái bẫy.** LLaMA Pro *thêm lớp* (32→40), MoD cho token *bỏ qua* block, LongLoRA đổi mẫu attention lúc train — thứ chạy ra **không còn là kiến trúc base**. A, B, C **giữ nguyên** đồ thị.

Đặt câu hỏi này trước

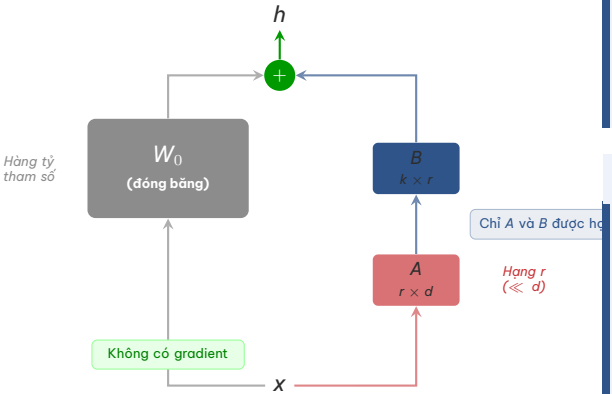
Trước khi thử một cái tên mới, hỏi **nó ở loại nào**:

A → tôi có đang làm *full* FT không?

B/C → đổi một dòng config, an toàn

D → tôi có sẵn sàng nuôi một kiến trúc *phi tiêu chuẩn* không?

Kiến trúc — Trọng số đóng băng + hai ma trận gầy



Khởi tạo — chi tiết quan trọng

$A \sim \mathcal{N}(0, \sigma^2)$ ngẫu nhiên, $B = 0$.

Nhờ $B = 0$ nên $\Delta W = 0$ tại bước đầu: model bắt đầu *đúng bằng* base model. Không có cú sốc chất lượng ở step 0.

Hệ số α/r

Tiền tố $\frac{\alpha}{r}$ giữ cho **độ lớn cập nhật trên mỗi thành phần hạng-1 không đổi** theo *r*.

Hệ quả (§11): learning rate tối ưu **gần như độc lập với rank**. Đổi *r* không bắt buộc phải dò lại LR.

Số tham số: $r(d + k)$ thay vì dk. Với $d = k = 4096$,

Bốn nút vặn, hai bậc tự do

Bạn tưởng mình có bốn siêu tham số độc lập:

$$\alpha, \quad \mathrm{LR}_A, \quad \mathrm{LR}_B, \quad \mathrm{init}_A$$

Nhưng động lực học **bất biến** dưới phép biến đổi:

$$\alpha \to \frac{\alpha}{pq}, \quad \mathrm{init}_A \to p \cdot \mathrm{init}_A,$$

$$\mathrm{LR}_A \to p \cdot \mathrm{LR}_A, \quad \mathrm{LR}_B \to q \cdot \mathrm{LR}_B$$

$\Rightarrow$ hai trong bốn nút là **dư thừa**. Không gian thật sự chỉ có **2 chiều**.

Lưu ý: Rất nhiều giờ GPU bị đốt để dò một lưới siêu tham số mà *một nửa số trục triệt tiêu lẫn nhau*. Hãy cố định α và init_A, chỉ dò learning rate và rank.

Quy ước thực dụng

Đặt $\alpha = 2r$ (tức $\alpha/r = 2$) rồi **không đụng đến nữa**.

Dò: `learning_rate` trước, `r` sau — và chỉ tăng r khi có bằng chứng thiếu năng lực (§11).

Nguồn: Thinking Machines Lab, *LoRA Without Regret* (2025)

Thinking Machines Lab, 2025

LoRA đạt **cùng hiệu suất mẫu** và **cùng chất lượng cuối cùng** như full fine-tuning khi thoả *hai* điều kiện:

(1) LoRA được gắn lên **mọi ma trận trọng số**, đặc biệt là các lớp **MLP / MoE**;

(2) số tham số huấn luyện được **vượt lượng thông tin chứa trong tập dữ liệu**.

Ra khỏi vùng này, chất lượng suy giảm **từ từ** theo mức thiếu hụt năng lực — không có “vách đá”. Vùng không hối tiếc phủ *phần lớn* các tình huống post-training thực tế.

Lỗi #1 — Chỉ gắn LoRA vào attention

Thí nghiệm trên Llama-3.1-8B, số tham số *gần bằng nhau*:

Vị trí gắn	Rank	Params	Kết quả
Chỉ attention	256	0.25 B	kém hơn
Chỉ MLP	128	0.24 B	**tốt hơn**

MLP thắng **dù ít hơn 0.01 B tham số**. Đây không phải sai số — đây là hiệu ứng vị trí.

Vì sao: dưới góc nhìn empirical NTK, xấp xỉ

$\mathrm{eNTK}(\mathrm{LoRA}) \approx \mathrm{eNTK}(\mathrm{FullFT})$ chỉ đúng khi LoRA phủ các lớp *chứa phần lớn tham số*. Trong một LLM, đó là MLP — không phải attention.

Đừng làm

```
target_modules=["q_proj","v_proj"]
```

Cấu hình mặc định trong rất nhiều tutorial 2023–2024 — và là nguồn gốc của phần lớn lời đồn "LoRA yếu hơn full FT".

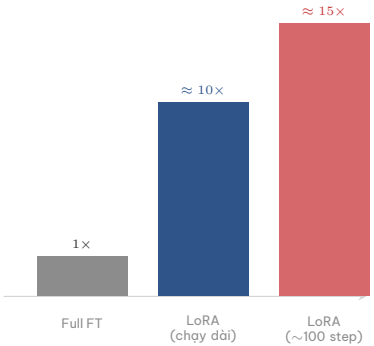
Hãy làm

```
target_modules="all-linear"
```

hoặc liệt kê đủ: `q,k,v,o_proj` + `gate,up,down_proj`

Nguồn: thinkingmachines.ai/blog/lora

Lỗi #2 — Dùng learning rate của full fine-tune



Learning rate tối ưu, tương đối so với full FT

Kết quả — LR tối ưu của LoRA **cao hơn full fine-tune khoảng 10 lần** — nhất quán ở cả supervised lẫn RL.

- Chạy ngắn (~100 step): nghiêng về **15×**, hội tụ về 10× khi train dài hơn
- Nhờ tiền tố α/r (§10), LR tối ưu **gần như độc lập với rank**

Lưu ý: Đây là lý do phổ biến nhất khiến một lần LoRA "không ăn thua": LR đặt theo thói quen full fine-tune, tức **thấp hơn tối ưu một bậc độ lớn**. Model học được rất ít, và ta đổ lỗi cho phương pháp.

Lỗi #3 — Tăng batch size để lấp đầy GPU

LoRA nhạy với batch size hơn full FT

Khoảng cách chất lượng LoRA–FullFT **nới rộng khi batch size tăng**, và điều này **độc lập với rank**.

Đây là đặc tính của cách tham số hoá tích ma trận BA — **tăng r không sửa được**.

Ảnh hưởng thực tế có giới hạn (cả hai phương pháp đều đạt loss tốt nhất ở batch nhỏ), nhưng đây là cái bẫy cho ai quen "tăng batch cho đầy VRAM".

Chi phí tính toán

Full FT (forward + backward): $3N^2$ phép nhân–cộng

forward N^2 · backward $2N^2$

LoRA: $2N^2 + 6NR$

Với $R \ll N$: **nhỉnh hơn $\frac{2}{3}$ của $3N^2$ một chút**.

Lưu ý: Đừng nhầm hai con số: $\frac{2}{3}$ là **FLOPs**. Khoản tiết kiệm **bộ nhớ** lớn hơn nhiều (§8) vì trạng thái optimizer co theo số tham số *huấn luyện được*, không phải tổng tham số.

Hoà giải hai kết quả tưởng như mâu thuẫn

VINUNIVERSITY

Biderman et al., 2024

"LoRA Learns Less and Forgets Less"

LoRA **kém hơn** full FT trên tác vụ đích...
...nhưng **giữ lại nhiều năng lực gốc hơn**.

Chống quên tốt hơn cả weight decay lẫn dropout; giữ được độ đa dạng sinh văn bản.

Thinking Machines, 2025

"LoRA Without Regret"

LoRA **bằng** full FT — với điều kiện gắn đủ lớp (kể cả MLP) và LR chia đúng thang ($\sim 10\times$).

Cả hai đều đúng — và đây là bài học

Hai kết quả không mâu thuẫn: phần lớn bằng chứng "LoRA học kém hơn" đến từ các thiết lập **chỉ gắn attention** và dùng **learning rate cỡ full fine-tune** — đúng hai lỗi ở §11.2 và §11.3. Sửa cấu hình thì khoảng cách *học* biến mất; nhưng lợi thế **chống quên** thì vẫn còn thật, và đó vẫn là một lý do chính đáng để chọn LoRA.

Bằng chứng 2026 — và hai bài báo tưởng như đối nghịch

A. Đội PEFT của HuggingFace (06/2026): *"LoRA không nên là mặc định tự động"*. Llama-3.2-3B, MetaMathQA → GSM8K:

Phương pháp	GSM8K	VRAM
Lily	**54,9%**	25,6 GB
LoRA (khởi tạo **rsLoRA**)	53,2%	22,6 GB
LoRA (chuẩn)	48,1%	22,5 GB
LoRA-FA	48,1%	20,2 GB
BEFT	32,9%	20,2 GB

Sinh ảnh (FLUX.2-klein-4B): **OFT 0,708 @ 9,01 GB** so với LoRA 0,697 @ 9,97 GB — *áp đảo trên cả hai trục.*

B. "Learning Rate Matters" (arXiv 2602.04998): quét siêu tham số cho **9 biến thể** + LoRA gốc trên toán, code, commonsense, instruction.

> *"Một khi learning rate được tinh chỉnh đúng, mọi phương pháp đạt đỉnh **trong khoảng 1–2%** của nhau"* — và **mỗi biến thể ưa một dải LR khác nhau**.

Các cải thiện đã công bố *"có thể không phản ánh ưu thế phương pháp"* — vì so sánh dùng siêu tham số cố định.

Hoà giải — và đây mới là bài học

Nhìn lại bảng A: **LoRA chuẩn 48,1% → 53,2% chỉ vì đổi cách chia thang** sang rsLoRA ($\alpha/\sqrt{r}$ thay cho α/r) — tức là đổi **learning rate hiệu dụng**. Chênh lệch lớn nhất trong "bảng vượt LoRA" **+5,1 điểm** là hiệu ứng LR; phương pháp lạ nhất (Lily) chỉ hơn baseline đã sửa **+1,7**. *Hai bài báo không mâu thuẫn — chúng xếp*

Hai thứ *thật sự* khác — và một cơ chế giải thích trần rank

VINUNIVERSITY

OFT / BOFT — cập nhật *nhân*, không phải *cộng*

LoRA: $W + BA$. OFT: $R \cdot W$ với R **trực giao** — giữ nguyên **hyperspherical energy** của trọng số gốc, tức chống quên bằng *cấu trúc*, không phải regularizer.

- Merge bình thường (`merge_and_unload`)
- `oft_block_size=32`, `use_cayley_neumann`
- Nhiều tham số hơn LoRA

GraLoRA — vì sao tăng rank hết tác dụng

Một cổ chai r duy nhất gây **rối gradient** giữa các kênh không liên quan — lý do chất lượng **chững hoặc tụt trên rank 32–64**.

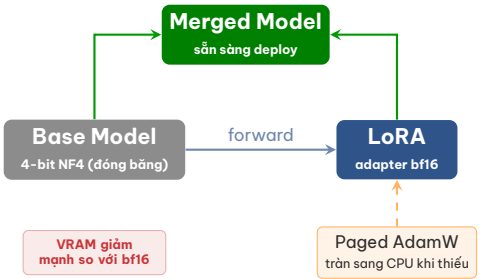
GraLoRA chia ma trận thành **khối con**, mỗi khối một adapter — chi phí "không đáng kể", tới **+8,5 Pass@1** trên HumanEval+.

Thứ tự thử — đã cập nhật

1. Sửa thang LR (và bật `rsLoRA`) — *miễn phí, hiệu ứng lớn nhất đo được* **2.** Khởi tạo tốt hơn: PiSSA / LoftQ (4-bit) / EVA **3.** *Rồi mới* đổi cơ chế: OFT (chống quên), GraLoRA (rank cao).

PEFT nay có hơn **20** phương pháp họ LoRA — học **các trục** thì đặt được bất kỳ tên mới nào vào đúng chỗ.

QLoRA — NF4 4-bit + adapter bf16



QLoRA (Dettmers et al., 2023) — Lượng tử hoá base xuống 4-bit NF4, gắn adapter bf16 lên trên. Chỉ adapter được huấn luyện.

- **NF4**: kiểu dữ liệu 4-bit tối ưu cho trọng số phân phối chuẩn — không phải int4 thông thường
- **Double quantization**: lượng tử hoá cả *hằng số lượng tử hoá* — tiết kiệm thêm
- **Paged optimizer**: đẩy trạng thái optimizer sang CPU RAM khi VRAM chạm đỉnh, thay vì OOM

Lưu ý: Gradient vẫn chảy *qua* base đã lượng tử hoá để tới adapter — base 4-bit

Thang chi phí — Full FT vs LoRA vs QLoRA (2026)

Phương pháp	Model ∼8B	Model ∼70B	Params huấn luyện
QLoRA (4-bit)	≥12 GB *(RTX 3060)*	∼48 GB *(1 GPU)*	∼1%
LoRA (bf16)	∼28 GB	>140 GB *(riêng trọng số)*	∼1%
Full FT (bf16+AdamW)	∼130 GB ⇒ đa GPU	∼1.120 GB	100%

Điều đáng nhớ là tỷ lệ

Full FT tốn **hơn 10**× QLoRA về bộ nhớ (16 byte/tham số, §8, chưa gồm activation). Con số GB tuyệt đối thay đổi theo từng thế hệ phần cứng và từng phiên bản thư viện — **hãy tự đo lại**, đừng chép bảng này vào báo cáo.

Lưu ý: **Rẻ nhất không còn tự động là đúng nhất.** Với vài model mới nhất (vd. dòng Qwen3.5), nhà cung cấp *khuyến nghị không dùng* QLoRA — sai số lượng tử hoá cao hơn bình thường — và đề xuất **LoRA bf16**, nhất là với biến thể MoE.

FSDP + QLoRA không loại trừ nhau: shard *và* lượng tử hoá cùng lúc — cách đưa 70B QLoRA về tầm một máy vài GPU.

RTX 5090 32GB: LoRA đến ∼13B fp16, hoặc QLoRA ∼32B. B200: 180 GB HBM3e. Nguồn: tổng hợp khảo sát phần cứng 2026 — xem RESEARCH

Ma trận QLoRA đa-bit — *nạp-và-train*, không phải *train-và-giao*

VINUNIVERSITY

Backend	Bit (train)	Hiệu chuẩn	Merge lại?	Ghi chú của tài liệu
bitsandbytes (=*LLM.int8*)	8, **4 (NF4)**	không	**có**	đường QLoRA chuẩn của PEFT
GPTQ (GPTQModel)	2, 3, 4, 8	**có** (≈20 ph/8B)	có	nhiều checkpoint dựng sẵn
HQQ	8/4/3/2/1	không	có	dưới 4-bit tụt rõ
EETQ	8 (int8)	không	có	tuyên bố nhanh hơn LLM.int8
AQLM	≈2	có (rất đắt)	**KHÔNG**	"không thể merge với trọng số AQLM"
torchao	**chỉ int8**	không	chỉ `int8_weight_only`	`int4` chưa hỗ trợ; NF4 chưa có
INC (Gaudi)	FP8	có	**KHÔNG**	`merge()` không hỗ trợ
Transformer Engine	FP8	—	**KHÔNG**	DoRA cũng không

Lưu ý: **Ba trong tám dòng không merge được**, dòng thứ tư chỉ merge đúng ở một dtype — ma trận liệt kê thứ bạn *nạp và train* được, không phải thứ bạn **giao đi** được (chính là cổng NB6 của lab). Thêm hai chi tiết: **AWQ không có** trong danh sách train của PEFT — nó là định dạng suy luận; và **không backend nào train ở 5 hay 6 bit** — chuỗi "2/3/4/5/6/8" là *hợp* của nhiều backend, 5/6-bit đến từ k-quant GGUF.

Bảng siêu tham số khởi điểm

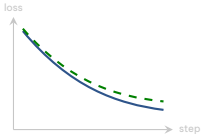
Tham số	Khởi điểm	Lý do / khi nào đổi
`target_modules`	`"all-linear"`	Phủ MLP là điều kiện của vùng không hối tiếc (§11.2)
`r`	16 | **256**	16 cho tác vụ hẹp; **256** khi dữ liệu ở quy mô post-train
`lora_alpha`	$2r$	Cố định rồi thôi — xem bất biến §10.3
`learning_rate`	$\approx 10\times$ LR full FT	Thang đúng quan trọng hơn giá trị tuyệt đối (§11.3)
LR schedule	cosine + warmup 5–10%	Warmup tránh sốc gradient ở bước đầu
`num_epochs`	1–3	>3 gần như luôn là quá khớp trên tập nhỏ
Batch hiệu dụng	$< \mathbf{32}$	LoRA kém dung nạp batch lớn — tăng rank *không* bù đ
`packing`	**tắt** nếu nhãn đã token hoá sẵn	Packing nối mẫu $\rightarrow$ phá căn chỉnh nhãn (§17.3)
`padding_free`	**chỉ khi** có FlashAttention *và* batch ≥ 2	FA-2 đòi **sm_80+**; T4 không có (§15.1)
`loss_type`	`"chunked_nll"`	Mặc định SFT của TRL từ v1.7 — giảm $\sim$30–50% VRAM

Lưu ý: Bảng này là **điểm xuất phát**, không phải đáp án — không gì thay được việc đo trên eval của chính bạn.

Đã thành tài liệu chính thức

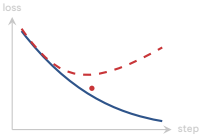
Khuyến nghị ở §11 nay nằm trong **tài liệu chính thức của TRL** (`lora_without_regret`), kèm script tái lập.

Đọc đường loss — ba hình dạng cần nhận ra



Lành mạnh

Cả hai cùng giảm rồi phẳng ra



Quá khớp

Train giảm, **eval quay đầu** ⇒ dừng ở điểm đỏ



Gai loss

LR quá cao, dữ liệu bẩn, hoặc tràn số ở bf16

Phản ứng đúng với từng hình

Quá khớp → giảm epoch, tăng dữ liệu, hoặc giảm rank — *không* phải giảm learning rate theo phản xạ. **Gai loss** → bật gradient clipping, kiểm tra vài mẫu quanh bước đó, xem lại LR. **Loss phẳng ngay từ đầu** → gần như luôn là **mask sai** (§17.2) hoặc LR thấp hơn tối ưu một bậc (§11.3).

Kernel *không phải* kiến trúc — hai trục khác nhau

§7.4 nói **model là gì** (CSA/HCA, Gated DeltaNet, KDA, local:global). Mục này nói **phép toán đó được chạy ra sao**. FlashAttention **không đổi** kết quả attention — nó đổi lượng dữ liệu phải đi qua HBM. Nhầm hai trục này là lỗi phổ biến khi đọc changelog.

Thang thế hệ FlashAttention — là hợp đồng phần cứng

Thế hệ	GPU tối thiểu	Ghi chú
FA-2	**Ampere sm_80+**	A100, 3090, 4090, L4
FA-3	Hopper (H100)	warp specialization, FP8
FA-4	Blackwell (B200)	CuTe-DSL, 03/2026
không có	**Turing sm_75**	**T4 — Colab Free**

FA-4 (arXiv 2603.05451, Tri Dao và cộng sự): tới **1613 TFLOPs/s** = 71% hiệu suất phần cứng trên B200; 1,3× so với cuDNN 9.13. Viết *hoàn toàn* bằng CuTe-DSL nhúng trong Python.

Lưu ý: `padding_free` cần kernel FlashAttention. Trên **T4** nó **không bao giờ** bật được — không phải vì cấu hình sai, mà vì sm_75 không có FA. Đây là lý do Lab 21 tắt cờ này thay vì bật một cờ vô tác dụng.

Quy tắc đọc

Toán tử tăng tốc = hợp đồng với phần cứng, **không phải** cờ cấu hình. Trước khi bật, hỏi: *GPU của tôi có compute capabil-*

Loss — lớp tốn bộ nhớ nhất không phải attention

Với từ vựng **150K–260K token**, tensor logits `[batch × seq × vocab]` là activation **lớn nhất** trong đồ thị — lớn hơn cả hidden state. Đây là chỗ "loss" thôi là công thức toán và trở thành hoá đơn VRAM.

Cut Cross-Entropy (Apple, ICLR 2025)

Chỉ tính logit của **token đúng**, còn log-sum-exp trên toàn từ vựng thì tính *on-the-fly* trong bộ nhớ nhanh — logits đầy đủ **không bao giờ tồn tại**.

Đo trên **Gemma 2**:

- bộ nhớ tính loss: **24 GB → 1 MB**
- classifier head: **28 GB → 1 GB**

Cùng một ý tưởng, ba cái tên bạn sẽ gặp

Tên	Ở đâu
`chunked_nll`	TRL — **bạn đã bật ở §14**
FLCE	Liger Kernel (LinkedIn)
CCE	`apple/ml-cross-entropy`

Liger hợp nhất **cả forward lẫn backward** của CE vào một kernel Triton: giảm $>4\times$ đỉnh bộ nhớ ở vocab 128K.

Vì sao điều này quan trọng với bạn

`loss_type="chunked_nll"` ở §14 — **đây là cơ chế đằng sau cờ đó**. Không phải mẹo vặt: nó là lý do một model 4B fine-tune vừa 16 GB.

Đọc ma trận tính năng — ba câu hỏi nó không trả lời

Trích nguyên văn README của LLaMA-Factory (08/2026) — và ba chỗ nó *im lặng*:

Mục	README nói	Ma trận im lặng về
Độ chính xác	16-bit full/freeze/LoRA + **2/3/4/5/6/8-bit** QLoRA qua AQLM/AWQ/GPTQ/LLM.int8/HQQ/EETQ	**merge được hay không** (§13)
Thuật toán	GaLore, BAdam, APOLLO, Adam-mini, Muon, OFT, DoRA, LongLoRA, LLaMA Pro, MoD, LoRA+, LoftQ, PiSSA	chúng là **bốn loại việc khác nhau** (§9)
Tăng tốc	FlashAttention-2, Unsloth, Liger Kernel, KTransformers, RoPE scaling, NEFTune, rsLoRA	**GPU tối thiểu** của từng cái (§15.1)
Engine	vLLM worker *hoặc* SGLang worker	—
Theo dõi	LlamaBoard, TensorBoard, Wandb, MLflow, SwanLab	—

Lưu ý: **Có mặt trong ma trận ≠ được dùng.** LongLoRA, LLaMA Pro, MoD vẫn được liệt kê — nhưng **không** tech report frontier nào trong tập nguồn của deck dùng chúng. Danh sách chứng minh *bảo trì*, không phải *áp dụng*.

Nó cũng *thay đổi*

So với danh sách phổ biến 2024, README nay thêm **APOLLO, Adam-mini, Muon, OFT**, **Liger Kernel, KTransformers** và **SGLang**. Tài liệu nào thiếu chúng là đã cũ.

Theo dõi thí nghiệm — giá trị nằm ở *cấu hình*

Bức tranh 2026

Công cụ	Vị trí
W&B	dashboard nhóm, cloud-first
MLflow	nguồn mở, self-host, registry
TensorBoard	debug cục bộ, thời gian thực
SwanLab	nguồn mở, giám sát cả phần cứng
LlamaBoard	*không cùng hạng* — GUI của LLaMA-Factory

Nhiều đội dùng **đồng thời**: TensorBoard để debug, MLflow làm registry, W&B cho cả nhóm nhìn.

Vì sao mục này nằm trong một buổi fine-tuning

Mọi lỗi ở §22 đều là run có **đường loss đẹp**. Giá trị của tracker không phải biểu đồ — mà là **cấu hình được ghi cạnh kết quả**: mask mode, LR, target modules, ngân sách step, seed.

Thí nghiệm bạn không dựng lại được thì không phải thí nghiệm — nó là một giai thoại có biểu đồ.

Lab 21 ép bạn làm bằng tay

`results/runs.csv` + cổng `max_steps` dùng chung. Bốn run chỉ so sánh được khi **cùng số step** — và

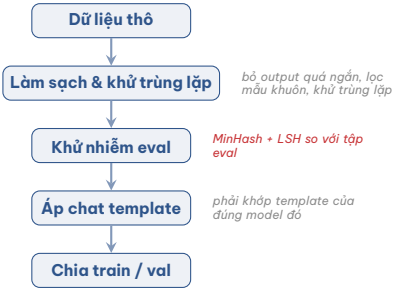
Thuê người khác fine-tune — bốn *kiểu* dịch vụ

Kiểu	Ví dụ (10/2026)	Bạn điều khiển	Tính tiền theo
Job công thức sẵn	Vertex (Gemini SFT; RFT preview — *chỉ thử nghiệm, không production*), Bedrock (Nova 2 Lite; RFT cho gpt-oss, Qwen3), Azure Foundry, Together	dữ liệu + vài siêu tham số	token train, hoặc giờ (RFT)
API huấn luyện lập trình được	Tinker, Fireworks Training API (GA 08/2026)	vòng lặp, loss, sampling, rank	token prefill / sample / train
GPU chạy code của bạn	HF Jobs, Modal, SageMaker training job, Anyscale	mọi thứ	giờ GPU
Đang đóng cửa	OpenAI self-serve (§1), Mistral hosted FT (deprecated), HF AutoTrain ("no longer maintained")	—	—

Một con số thật: Gemini 3.5 Flash SFT/RFT **$10 / 1M token train**; 3.1 Flash-Lite SFT **$3**. Bedrock RFT trên model mở dùng **LoRA rank 32**.

Lưu ý: "Có trong catalog" ≠ "tune được ở đó". Và "managed" có thể chỉ là *thuê GPU* — hỏi ai viết vòng lặp huấn luyện.

Quy mô dữ liệu — chất lượng đè bẹp số lượng



Cần bao nhiêu mẫu?

- Phong cách / định dạng: **500–2.000** mẫu chất lượng cao
- Thích nghi miền: **10.000+** mẫu
- Quy mô continued-pretraining: **tỷ token** — lúc đó xem lại §8 và §6

> **Quy tắc — 500 mẫu hoàn hảo > 10.000 mẫu nhiễu.** Một mẫu sai dạy model một thói quen sai — và nó học rất nhanh.

Vì sao ít mà tinh lại thắng: model đã biết ngôn ngữ. Bạn không dạy nó tiếng Việt — bạn chỉ cho nó thấy *dạng câu trả lời bạn muốn*. Vài trăm ví dụ nhất quán làm việc đó tốt hơn hàng vạn ví dụ mâu thuẫn nhau.

Che loss — nút vặn quan trọng nhất mà ít ai dạy

```python
from trl import SFTTrainer, SFTConfig

cfg = SFTConfig(
    max_length=2048,           # KHONG phai max_seq_length
    packing=True,
    packing_strategy="bfd",    # best-fit-decreasing
    padding_free=True,         # can FlashAttention (S15)
    completion_only_loss=True, # prompt/completion data
    # assistant_only_loss=True # multi-turn: needs {%
          generation %}
)
trainer = SFTTrainer(
    model=model, args=cfg,
    train_dataset=ds,
)
trainer.train()
```

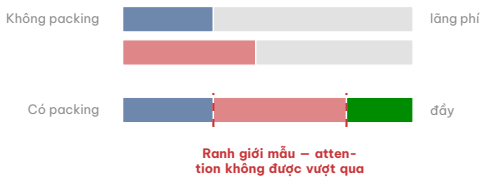
Lưu ý: Loss trên *cả* prompt = dạy model **sinh câu hỏi của bạn**. Muốn học câu trả lời: chỉ tính loss trên câu trả lời.

Nghi thức kiểm tra mask

Giải mã ngược các vị trí `mask == 1`; phải *đúng bằng* nội dung assistant. 5 phút, cứu một run 8 tiếng.

`assistant_only_loss` cần thẻ `{% generation %}`; Qwen3.5 gốc thiếu ⇒ mask **rỗng** (Lab F-10). TRL ≥1.10 tự vá hoặc **báo lỗi**; mask *tự dựng* thì im lặng.

Sequence packing — tăng tốc miễn phí, *có điều kiện*



Packing — Ghép nhiều mẫu ngắn vào một chuỗi dài để không lãng phí token padding. Thông lượng tăng đáng kể.

Lưu ý: **Nhiễm chéo:** nếu attention được phép nhìn xuyên ranh giới, mẫu B "đọc trộm" mẫu A. Model học được mối liên hệ không tồn tại, và bạn sẽ không thấy điều đó trong loss.

Cách sửa: `padding_free=True` + kernel attention hiểu độ dài thay đổi (FlashAttention varlen), để position id được reset ở mỗi ranh giới.

Fine-tune model biết suy luận — cái bẫy không báo lỗi

Model mở 2026 đều có chế độ suy luận (§2). Fine-tune bằng dữ liệu *hỏi → đáp* thông thường sẽ **phá huỷ suy luận trong khi accuracy vẫn tăng** — không chỉ số quen thuộc nào báo động.

Model / tác vụ	Cách biểu diễn	Trace hợp lệ
Qwen3-8B, hoá học	khối rỗng	**0%**
Llama-R1-8B, GSM8K	bỏ hẳn khối	58,2%
Llama-R1-8B, GSM8K	khối rỗng	**100%**
Olmo-3-7B	cả hai	trace *bị cụt*

Qwen3-8B vẫn đạt 56,6% accuracy — vừa học việc mới, vừa mất sạch suy luận.

Chiều tác động **phụ thuộc model**.

Liều thuốc: lại là che loss

masked-think — bỏ vùng suy luận rỗng khỏi loss.

response-only — che prompt *và* vùng suy luận, chỉ tính loss trên đáp án.

Cả hai **không cần trace của thầy**. Distillation cũng được — nhưng **hỏng với Olmo-3-7B**.

Quy tắc

Base có chế độ suy luận ⇒ **chủ động quyết định** khối suy luận nằm ở đâu trong loss, và thêm chỉ số "tỷ lệ trace hợp lệ" vào bộ eval (§21).

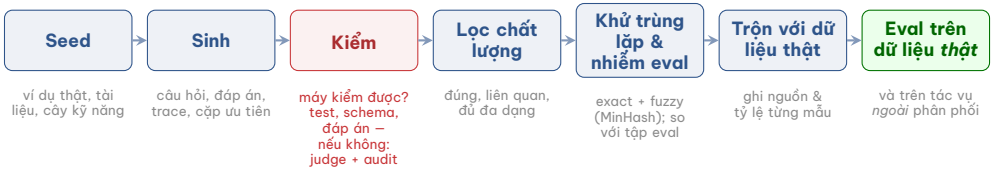
Dữ liệu tổng hợp là một *họ* phương pháp

Bắt đầu từ…	Kỹ thuật	Cơ chế một dòng	Sinh ra
Vài ví dụ mẫu	Self-Instruct, Alpaca	175 cặp mẫu → model lớn viết thêm → 52K	câu hỏi + đáp
Câu lệnh đơn giản	Evol-Instruct	viết lại từng bước cho *khó hơn*	bài khó dần
Template chat rỗng	Magpie	chỉ đưa phần đầu template, model tự "đóng vai" user	hội thoại
Persona / cây kiến thức	Persona Hub, GLAN	đổi người hỏi, đổi chủ đề để *phủ rộng*	độ phủ
Văn bản thật	back-translation	coi văn bản là đáp án, đoán câu hỏi sinh ra nó	cặp từ dữ liệu thật
Teacher biết suy luận	distill trace (s1, OpenThoughts)	ghi lại chuỗi suy luận của model lớn	chuỗi suy luận
Bộ kiểm tự động	rejection sampling (STaR)	sinh *K* đáp án, giữ cái qua test	đáp án đã kiểm

Câu hỏi đầu tiên: dữ liệu này dạy *cái gì*?

Nemotron-CC, **Cosmopedia**: dữ liệu *pre-training* (kiến thức). **SmolTalk**, **Tülu 3**: dữ liệu *SFT* (cách trả lời) — loại người fine-tune cần.

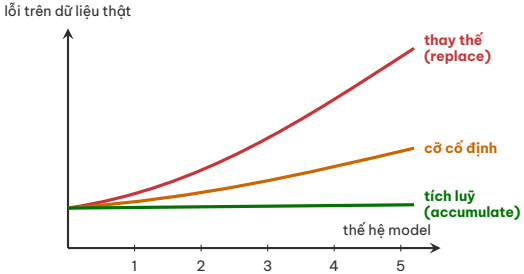
Đường ống sinh dữ liệu — và bao nhiêu mẫu sống sót



Dự án	Giữ lại	Cách lọc
AlpaGasus	9K / 52K ≈ 17%	LLM chấm điểm
Magpie	300K / 4M ≈ 7,5%	lọc nhiều tiêu chí
FROST (09/2026)	bỏ 20–30%	gradient trên dữ liệu thật
Tülu 3	bỏ *cả bộ* nếu trùng eval >2%	khử nhiễm

Lưu ý: **Không có tỷ lệ giữ "chuẩn".** 17% và 7,5% là của hai dự án khác nhau. Thói quen đúng: **ghi số dòng sau mỗi bước** — và đọc ngẫu nhiên cả những mẫu *bị loại*.

Model collapse — câu chuyện đúng



minh hoạ định tính theo Kazdan et al., ICML 2025

Điều đã được chứng minh (Nature, 2024): train *đệ quy* trên output của chính mình, qua nhiều thế hệ, làm mất dần **đuôi phân phối** — những trường hợp hiếm.

Điều quyết định: *thay thế* dữ liệu thật thì sụp; *tích luỹ* thêm bên cạnh dữ liệu thật thì loss ổn định.

> **Lưu ý:** **Ngộ nhận:** "một lần SFT trên dữ liệu tổng hợp đã lọc gây model collapse". Các bài báo *không* cho thấy điều đó.

Rủi ro thật ở quy mô của bạn: quên năng lực ngoài phân phối, và thừa hưởng nhiễm benchmark từ teacher (§17).

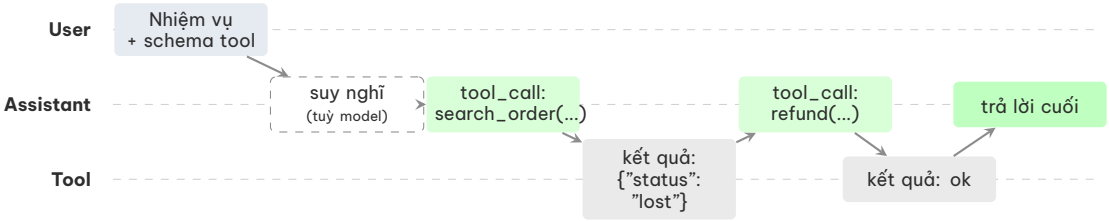
Ai cho phép bạn học từ output của họ?

Nguồn dữ liệu (teacher)	Điều khoản về việc train model khác — bản kiểm 10/2026
OpenAI (Terms, 01/2026)	cấm dùng output để phát triển model *cạnh tranh*; hợp đồng API có "Permitted Exception" (vd. fine-tune trong chính dịch vụ OpenAI)
Gemini API (03/2026)	"không được dùng dịch vụ để phát triển model cạnh tranh"
Anthropic (03/2026)	được train model *không* cạnh tranh (công cụ chuyên biệt); chatbot đa năng cần phép bằng văn bản
DeepSeek-R1 (MIT)	cho phép **rõ ràng**: "distillation for training other LLMs"
Llama 3	**cấm** dùng output để cải thiện LLM khác — điều khoản này **đã bỏ từ Llama 3.1**
Llama 3.1 trở đi (gồm Llama 4)	cho phép, nhưng tên model phân phối phải **bắt đầu bằng "Llama"**
Gemma 4, Muse Glimmer, gpt-oss (Apache 2.0)	quyền sửa đổi chung; không nêu riêng chữ "distillation"

Thói quen bắt buộc: provenance từng mẫu

Ghi **teacher, phiên bản điều khoản, ngày sinh** vào từng dòng dữ liệu — NVIDIA gắn license theo từng mẫu. "Tôi sở hữu output" $\neq$ "tôi được dùng nó để train mọi thứ". Pháp lý đầy đủ: §24.

Trajectory là gì — một ván chơi được ghi lại từng nước



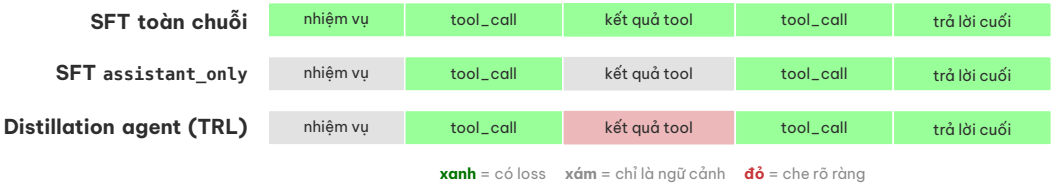
Cùng một ván, ba cách viết ra token:

- TRL: `messages` + `tools`, lượt `tool_calls` và vai `tool`
- OpenAI API: sự kiện function call có `call_id`
- Hermes / Qwen3: thẻ `<tool_call>...</tool_call>` trong văn bản

Trường suy nghĩ cũng khác nhau: `reasoning_content` (Qwen), `thinking` (Gemma).

Lưu ý: Dữ liệu ghi bằng định dạng tool của model A, train cho model B dùng định dạng khác ⇒ model học *sai giao diện*. Luôn render qua chat template của **chính** model bạn train.

Ngữ cảnh ≠ nhãn — model *thấy* gì và *học* gì



Hàng 1 còn dạy model *dự đoán* kết quả tool — một mục tiêu khác. Muốn bắt chước *hành động* agent: loss trên tool-call/trả lời, tool output giữ làm *ngữ cảnh*. Đo tác động trên hành vi thật.

"Tool output luôn bị che" là **sai** như một quy tắc chung: TRL để `assistant_only_loss=False` mặc định; chỉ đường distillation agent mới tự che kết quả tool.

Lưu ý: **Nhớ F-10 (§17):** mask lấy từ tokenizer **rỗng** khi template thiếu `{% generation %}`. Với trajectory còn nặng hơn: một mask sai che cả tool-call lẫn trả lời cuối — giải mã ngược *một trajectory đầy đủ* trước khi train.

Trajectory từ đâu ra — và đếm *đúng đơn vị*

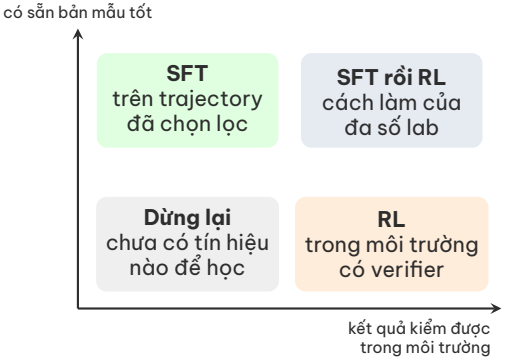
Dự án	Con số công bố	Đơn vị thật sự là gì
APIGen (2024)	3.673 API → 60K mẫu	mẫu gọi hàm, qua 3 lớp kiểm: định dạng, chạy thật, ngữ nghĩa
APIGen-MT (2025)	5.000 hội thoại	nhiều lượt, user mô phỏng; tạo task 28%→70% đạt nhờ phản hồi LLM
SWE-Gym (2024)	2.438	*task* Python có môi trường + test — chưa phải trajectory
SWE-smith (2025)	50K task → 5.000	*trajectory thành công*, tối đa 3 / task, trung bình ∼58 lượt
R2E-Gym (2025)	3.321	trajectory thành công từ 2.048 môi trường (bộ khác cho verifier)
SWE-rebench V2 (2026)	32.079	task chạy được, 20 ngôn ngữ — *chưa* có lần chạy nào

Task ≠ trajectory ≠ mẫu train

Task là đề bài. Trajectory là *một lần thử*. Mẫu train là lần thử *đã qua bộ lọc*. Ba con số này có thể chênh nhau 10 lần.

Công thức chung của các pipeline: tạo task + schema → teacher chạy thử (hoặc user mô phỏng) → **thực thi và kiểm** → giữ lần thành công. Giống hệt §18 — chỉ khác là *bộ kiểm* giờ là môi trường chạy thật.

SFT trên trajectory, hay RL trong môi trường?



SFT khi hành vi lặp lại và có bản mẫu đáng tin. **RL** khi thành công phụ thuộc trạng thái thay đổi, chuỗi hành động dài, và kết quả *kiểm được*.

Môi trường giờ là dữ liệu:

- Prime Intellect Environments Hub (08/2025): môi trường = "state, action, reward"
- OpenEnv (HF, 06/2026): terminal, trình duyệt… cho agentic RL
- MiMo-V2.6 (09/2026): một lần GRPO trên >**7.000** môi trường; Granite 4.2 (08/2026): RL trong sandbox thật

Verifier là một phần của dữ liệu

> Nó định nghĩa "thành công". Verifier lỗi hoặc dễ lách ⇒ RL dạy đúng cái lỗi đó (ví dụ thật ở §20).

Công cụ quen thuộc — dữ liệu và môi trường tạo khác biệt

> *"All substantive changes lie instead in the data pipeline."*
>
> — báo cáo kỹ thuật DeepSeek-V4.1-Flash (09/2026), nói về post-training của *chính họ* so với bản trước: vẫn SFT → RL → on-policy distillation.

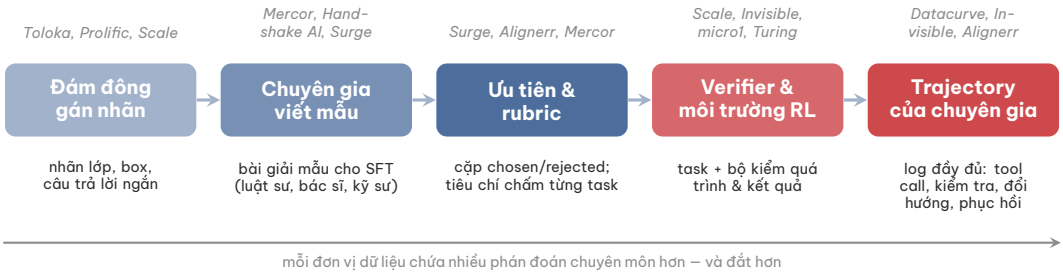
Cái họ đã đổi: tự động *sinh task* có lời giải tham chiếu và tín hiệu thưởng, *dựng môi trường* theo quy trình, rồi lọc, khử trùng lặp và hiệu chỉnh độ khó.

Release 08–09/2026	Công thức post-training
DeepSeek-V4.1-Flash	SFT → RL → OPD
MiMo-V2.6	một lần GRPO, >7.000 môi trường
Qwen3.8-Omni	chuyên gia → distill → RL chung
SEA-LION v4.8	SFT + online OPD
Muse Glimmer	SFT + OPD + RL
Granite 4.2	SFT trên trajectory + RL sandbox

Hệ quả cho bạn

Cùng bộ đồ nghề (§5–§6), thứ tự *khác nhau*. Lab lớn thắng nhờ **task, môi trường, verifier, trajectory**; dữ liệu *miền của bạn* thì họ không có.

Chuỗi giá trị dữ liệu huấn luyện 2026

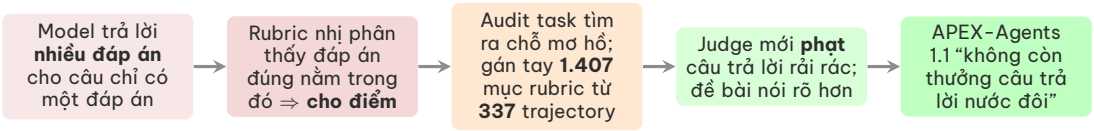


Đọc theo kỹ thuật, không theo công ty: cùng một công ty thường bán ở nhiều chặng. Câu hỏi đúng là "tôi cần *loại tín hiệu* nào?" — không phải "thuê ai?".

Lab vẫn giữ quyền định nghĩa

Nhà cung cấp tìm và quản lý người; **lab quyết định thế nào là câu trả lời tốt**. InstructGPT: 40 contractor, chọn và hướng dẫn bởi chính OpenAI.

Rubric cũng bị "lách" — câu chuyện scattergunning

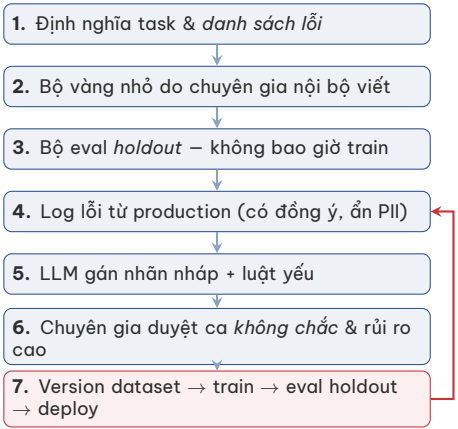


Vì sao chuyện này là chuyện của bạn: nếu bạn dùng rubric hay LLM judge để *lọc dữ liệu* (§18) hoặc làm *reward* cho RL (§19), model sẽ học đúng lỗ hổng của nó. Sửa prompt mà không sửa bộ chấm thì lỗ hổng vẫn còn.

Luật rõ ràng thắng tiền thưởng

Thí nghiệm 307 người gán nhãn: hướng dẫn **rõ ràng** chính xác hơn hướng dẫn mơ hồ **14 điểm**; rõ ràng + thưởng đạt 87,5%. Viết hướng dẫn tốt rẻ hơn trả thêm tiền.

Data engine cho một team nhỏ — không cần mua dữ liệu



> **Data-centric AI** — "Kỷ luật *kỹ thuật hoá có hệ thống* dữ liệu cần để xây hệ AI" (Andrew Ng, 2021) — một **phương pháp**, không phải một ngành hàng.

Bằng chứng có điều kiện: trong DCLM, giữ phần lớn công thức train cố định và chỉ đổi bộ lọc thì lọc bằng model thắng rõ. Model 7B cuối đạt 64% MMLU, hơn MAP-Neo **6,6 điểm** với **ít hơn 40%** compute (hai công thức khác nhau). Nhưng ở SFT cỡ triệu mẫu, chọn ngẫu nhiên gần như ngang các bộ lọc "chất lượng" (§18).

Với tiếng Việt: dữ liệu miền của bạn là thứ không lab nào có — và bộ eval nên do *người bản ngữ trong nghề* viết.

Vì sao perplexity một mình là chưa đủ

Perplexity đo cái gì

Mức "bất ngờ" của model trước văn bản trong tập của bạn.

PPL giảm nghĩa là model **giống tập dữ liệu của bạn hơn** — kể cả khi tập đó dở hoặc thiên lệch.

Một model học thuộc lòng tập train có PPL rất đẹp.

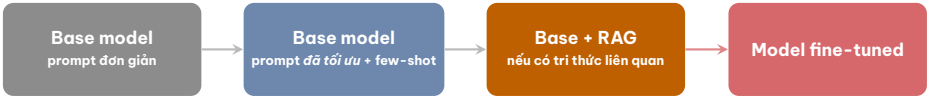
Nên đo bốn nhóm

1. **Tác vụ đích** — chính xác trên việc bạn thuê model làm
2. **Hồi quy năng lực** — benchmark tổng quát trước/sau, bắt quên thảm hoạ
3. **Tuân thủ định dạng** — tỷ lệ JSON/schema hợp lệ
4. **Vận hành** — độ trễ, token/giây, chi phí mỗi yêu cầu

Cổng hồi quy

Trước khi train, **đóng băng** tập eval và ghi điểm base model. Sau khi train, chạy lại đúng tập đó. Nhóm (2) tụt quá ngưỡng ⇒ **không deploy**, bất kể nhóm (1) đẹp đến đâu. Eval đầy đủ: Ngày 14.

Ba baseline phải so — RAG khi bài toán cần



Lưu ý: So bản fine-tune với **base model + prompt tệ** là cách tự lừa mình phổ biến nhất. Gần như lần nào bạn cũng "thắng" — và không học được gì.

Câu hỏi kết luận

Bản fine-tune có thắng **base model đã được prompt tử tế** một khoảng đủ lớn để bù cho chi phí huấn luyện, phục vụ và bảo trì nó không?

Bảng tra triệu chứng

Triệu chứng	Nguyên nhân hay gặp nhất	Kiểm tra
Model **không bao giờ dừng** sinh	EOS lúc train $\neq$ EOS lúc suy luận	So `eos_token` với token template phát r
Model *viết tiếp* câu hỏi của bạn	Loss tính cả trên prompt	Bật `completion_only_loss` (§17.2)
Đầu ra lặp / vô nghĩa	Sai chat template hoặc special token	Giải mã ngược một mẫu đã tokenize
Loss phẳng ngay từ step 0	Mask sai, hoặc LR thấp một bậc	Kiểm tra mask; nhân LR lên 10
Điểm eval đẹp, production tệ	Trôi phân phối / nhiễm benchmark	Eval trên đầu vào người dùng thật
Giỏi miền mới, dốt mọi thứ khác	Quên thảm hoạ	Thêm 1–5% replay (§6.3)
Gai loss rồi NaN	LR quá cao / tràn số	Gradient clipping; kiểm tra dữ liệu
Mất suy luận, accuracy vẫn tăng	Template nuốt khối `<think>`	In chuỗi *sau* `apply_chat_template` (§17.

Lưu ý: Không lỗi nào ở trên làm training **dừng lại**. Tất cả đều chạy đến hết, in ra một đường loss trông hợp lý, rồi giao cho bạn một model hỏng. Đây là lý do phải kiểm tra *trước* khi train.

Ca bệnh điển hình — lệch chat template

Sự cố có thật

Một cấu hình Qwen3 base đặt `eos_token` thành `<|endoftext|>`, trong khi chat template vẫn phát ra `<|im_end|>`.

Hai token khác nhau cho cùng một khái niệm "kết thúc lượt".

IFEval: 67,84 → 41,96.

Không có thông báo lỗi nào. Training chạy bình thường.

Quy tắc vàng

Áp **đúng một** chat template lúc huấn luyện và lúc suy luận. Nếu khác nhau, model của bạn kỳ vọng một định dạng mà *không ai* gửi tới.

Danh sách kiểm tra 3 dòng

1. `tokenizer.eos_token` là gì?
2. Template kết thúc lượt bằng token nào?
3. Hai cái đó có **giống nhau** không?

Khi fine-tune một **base** model bằng dữ liệu hội thoại, bạn thường *phải* đổi `eos_token` cho khớp template — base model chưa có khái niệm "hết

Hai đường triển khai — gộp hay giữ riêng

Gộp (merge)

$W = W_0 + \frac{\alpha}{r} BA$ — adapter tan vào trọng số gốc.

Được: đồ thị suy luận *giống hệt* base model $\Rightarrow$ **0% overhead**; xuất được GGUF/AWQ; chạy được ở mọi backend.

Mất: một model đầy đủ cho *mỗi* biến thể — *n* khách hàng là *n* bản sao.

Giữ riêng (multi-LoRA)

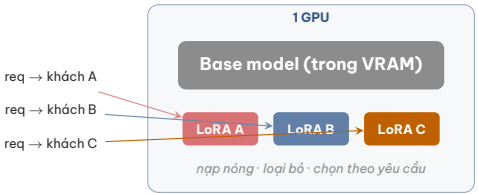
Một base model trong VRAM + nhiều adapter nạp theo yêu cầu.

Được: *n* khách hàng trên **một** GPU; thêm/bớt adapter nóng, không khởi động lại.

Mất: một ít overhead mỗi yêu cầu; backend phải hỗ trợ.

Lưu ý: DoRA: vector độ lớn phải được gộp đúng cách — cần PEFT $\geq$ 0.10. Với phiên bản cũ hơn, model sau khi merge *không* bằng model lúc train, và bạn chỉ phát hiện ra khi eval sau merge tụt điểm.

Multi-LoRA — kinh tế học của một base, nhiều adapter



- vLLM cho phép cập nhật adapter lúc chạy qua `VLLM_ALLOW_RUNTIME_LORA_UPDATING` cùng endpoint nạp/gỡ
- **S-LoRA** báo cáo **tới 4**× thông lượng so với hỗ trợ LoRA ngây thơ, và nhiều bậc độ lớn về số adapter đồng thời, nhờ batching không đồng nhất + kernel CUDA riêng
- Cô lập khách hàng nằm ở tầng định tuyến, không ở tầng model

Vì sao đây là lập luận kinh doanh

Fine-tune riêng cho từng khách hàng thường bất khả thi vì *chi phí phục vụ*, không phải chi phí huấn luyện. Multi-LoRA xoá đúng khoản đó.

Chi tiết stack phục vụ: **Ngày 20**.

License đi theo bản fine-tune của bạn

Họ model	License	Hệ quả khi bạn fine-tune
Qwen3 (phần lớn)	Apache 2.0	Fine-tune, thương mại hoá, phát hành lại — chỉ cần ghi công
Gemma 4	Apache 2.0 *(từ 04/2026)*	Như trên; vẫn còn Prohibited Use Policy riêng
Muse Glimmer (Meta)	**Apache 2.0** *(08/2026)*	Chính Meta nay cũng phát hành giấy phép dễ dãi
DeepSeek-V4, GLM-5	MIT	Ràng buộc lỏng nhất trong nhóm
Kimi K3	License riêng	**Phải đọc toàn văn** trước khi thương mại hoá
Llama	Community License	Bản fine-tune **là** Llama derivative — xem cảnh báo dưới

Điều khoản âm thầm phá pipeline

License Llama cấm dùng **đầu ra hoặc bản fine-tune của Llama để huấn luyện / chưng cất một model *không phải* Llama**.

Nghĩa là: sinh dữ liệu tổng hợp bằng Llama rồi train Qwen là **vi phạm**.

Hạ nguồn cũng thừa kế AUP, ngưỡng 700M MAU và điều khoản này.

Câu hỏi cần trả lời trước khi chọn base

1. Tôi được phép thương mại hoá không?
2. Tôi có phải phát hành lại dưới cùng license không?
3. Đầu ra của model này có được dùng để train model khác không?
4. Có ngưỡng người dùng kích hoạt điều khoản mới không?

Chọn base model là quyết định **pháp lý** ngang với kỹ thuật. Và bản đồ này *đổi theo quý* — kiểm tra lại license mỗi lần khởi động dự án.

Bốn thứ hỏng mà loss không hề báo

LIVE DEMO

1. **Mask sai:** giải mã ngược `labels` — mask lấy từ tokenizer giám sát **0 token** trên template không có `{% generation %}` — TRL báo lỗi, pipeline tự viết thì im lặng
2. **Lệch template:** train một dạng prompt, chấm một dạng khác — loss đẹp, điểm đích **0,000**
3. **Thang learning rate:** cùng dữ liệu, LR full-FT so với LR $\times 10$ — hai đường loss khác hẳn
4. **Xếp hạng bằng loss:** run `attn_only` có loss huấn luyện *thấp hơn* run đúng — và vẫn là cấu hình tệ hơn

LAB #21

Mục tiêu: Fine-tune một open-weight model bằng LoRA — và chứng minh nó thắng được base model đã prompt tử tế

Deliverable: Adapter LoRA + báo cáo đối chiếu **ba baseline**, **bốn nhóm điểm**, kèm ba run đối chứng cùng ngân sách step

Thời gian: ≈2 giờ trên Colab Free T4

Lab 21 — năm notebook, và cái mà mỗi cái ép bạn chứng minh

NB	Tên	Phải chứng minh điều gì	T4 (đo thật)
1	data & mask	phần được tính loss **đúng bằng** câu trả lời — giải mã ngược	≈25 giây
2	baselines	mốc được **đóng băng TRƯỚC** khi train: (a) prompt ngây thơ, (b) prompt đã tối ưu	17–23 ph
3	train correct	cấu hình vùng-không-hối-tiếc, in `layer_types` của chính model	15–25 ph
4	misconfig autopsy	3 run đối chứng **cùng số step**: `attn_only` · `wrong_lr` · `qlora`	45–60 ph
5	evaluate & verdict	4 nhóm + **cổng hồi quy** — và một phán quyết có thể là *THẤT BẠI*	≈21 ph

Cấu hình `correct` — vì sao khác §14

`all-linear` · r=16 · α=2r · LR ≈10× · batch <32 · `chunked_nll`

LoRA bf16/fp16, KHÔNG phải QLoRA — 4-bit là một *run đối chứng*. `packing` tắt; `padding_free` không dùng được (§15.1).

Lưu ý: Bốn nhóm đều có thang **khách quan** — không LLM judge, nên không có "điểm cho không": **target** · **regression** (15 câu phổ thông) · **format** (JSON đủ 4 khoá) · **latency**.

Điểm nằm ở chỗ bạn *biết*, không phải ở chỗ bạn *thắng*

Baseline (b) là một cái bar thật

Bản fine-tune phải thắng **base model đã được prompt tử tế** — không phải thắng base model bị prompt cẩu thả. Rất nhiều báo cáo "fine-tune thành công" chỉ thắng được (a).

`make verify` kiểm tra phép so sánh bạn được chấm là **công bằng**: checksum eval, ngân sách step dùng chung, artefact đủ.

Hai cần gạt khi bị bó thời gian

`EVAL_LIMIT=8` → rút ngắn phần **sinh**

`EPOCHS=1` → nửa số **step**

Đo thật: cả hai bật, NB1+2+3+5 hết **17 phút**. Để **mặc định** khi nộp bài.

`EPOCHS` áp cho **cả** NB3 lẫn NB4 — không chỉnh được một nửa. Ba run đối chứng chỉ có nghĩa khi chạy đúng bằng số step của `correct`.

Phát hiện đắt giá nhất của lab này

Một lần chạy thật đã cho **cả bốn** adapter điểm target = 0,000 — trong khi loss hoàn toàn khoẻ mạnh. Nguyên nhân: train một dạng prompt, chấm một dạng khác. **Một mask loss đã kiểm chứng không bảo đảm một prompt đúng** — đó là hai trục khác nhau.

Tổng kết — Key Takeaways

Những ý chính cần nhớ trước khi sang bài tiếp theo

1. **Quyết định:** leo bậc thang baseline; chỉ đổi trọng số khi đo được khoảng cách
2. **Model đến từ đâu:** SFT, RL, distillation là đồ nghề chung; khác biệt nằm ở *dữ liệu và môi trường*
3. **Lõi:** LoRA đủ dung lượng, phủ mọi lớp, dò đúng LR thì gần full FT — xác nhận bằng eval
4. **Dữ liệu:** sinh thì rẻ, *kiểm* mới đắt — verifier, provenance, eval trên dữ liệu thật
5. **Ra sản phẩm:** bộ hồi quy & an toàn; adapter phụ thuộc base, license, endpoint

Bài tiếp theo

Ngày 22: DPO, ORPO & Alignment

"Hôm nay ta dạy model cách trả lời. Ngày mai: dạy nó trả lời tốt hơn theo sở thích con người — với RL, rank-1 adapter đã đủ; DPO có giống vậy không?"

Bài tập về nhà

- Hoàn thành Lab 21 — kèm bảng ba baseline
- Đọc: Rafailov et al., "Direct Preference Optimization" (2023)
- Đọc (khuyến nghị): Thinking Machines Lab, "LoRA Without Regret" (2025)

Hỏi & Đáp

Use case của bạn thật sự cần fine-tune, hay chỉ cần một prompt tử tế hơn? Hãy mang một ví dụ cụ thể ra mổ xẻ.



Cảm ơn!

AICB-P2T3 · Ngày 21 · Fine-tuning LLMs
— Từ Full Fine-tune đến LoRA/QLoRA

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn